

Практическая работа №2

Выполни: Смирнов Андрей Юрьевич

Группа: ИКБО-66-23

Цель работы

Освоить работу с базовыми компонентами Unity (Transform, Rigidbody, Collider), создать пользовательский контроллер игрока с переключением камеры и реализовать навигацию NPC с использованием NavMesh.

Задачи работы

1. Продемонстрировать работу компонентов Transform, Rigidbody и Collider на примере взаимодействия объектов сцены.
2. Реализовать пользовательский контроллер игрока с возможностью:
 - перемещения,
 - прыжка,
 - переключения камеры от первого и третьего лица,
 - выполнения дополнительного действия (ускорение или приседание).
3. Настроить навигационную систему NavMesh, создать NPC, реализовать его перемещение по NavMesh и разделить навигационную сетку на зоны с разной скоростью движения.

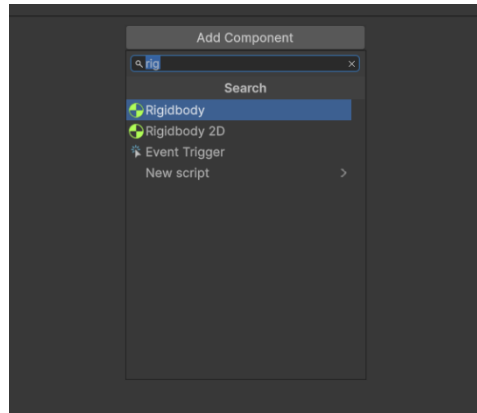
1. Подготовка сцены

Создайте новый Unity проект и откройте сцену SampleScene. Добавьте на сцену объект Plane (GameObject - 3D Object - Plane) – это будет поверхность уровня.

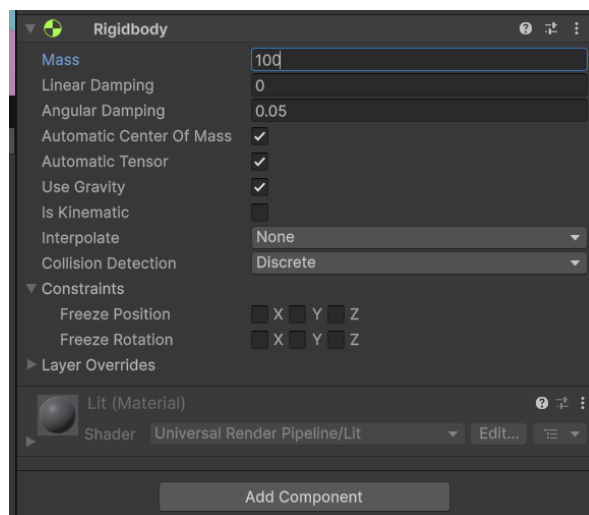
Для демонстрации физики добавьте объект Sphere:

GameObject - 3D Object – Sphere

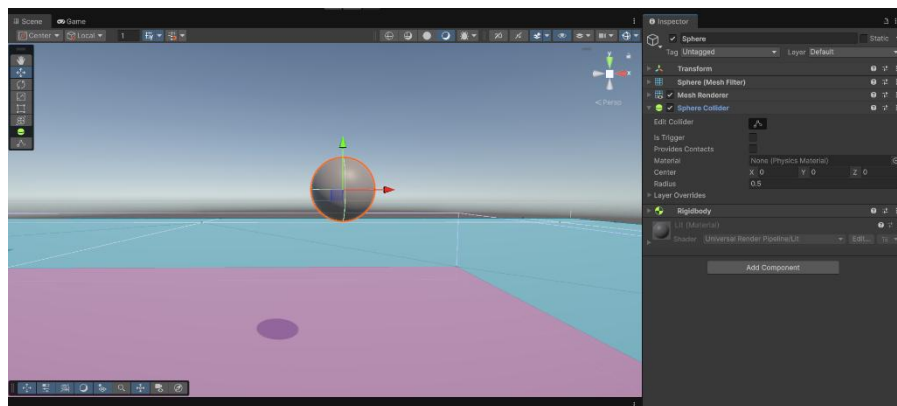
Для объекта Sphere добавьте компонент Rigidbody.



Включите параметр Use Gravity и убедитесь, что Is Kinematic отключен.



После запуска сцены сфера будет падать и взаимодействовать с коллайдерами.



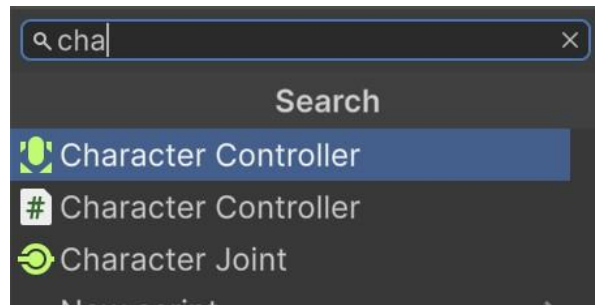
2. Создание игрока

Создайте объект Capsule:

GameObject - 3D Object – Capsule

Переименуйте объект в Player.

Добавьте компонент Character Controller.

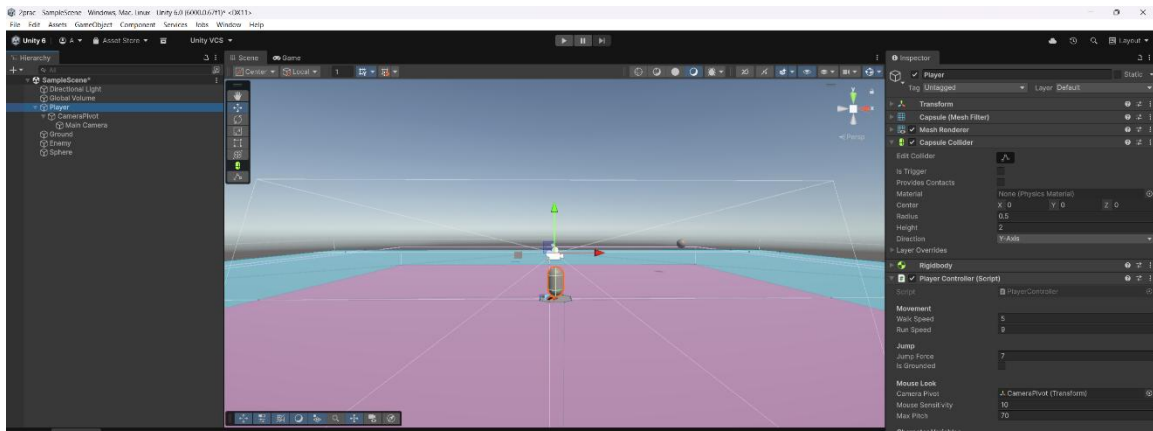


Создайте две камеры:

Camera_FirstPerson

Camera_ThirdPerson

Сделайте их дочерними объектами Player.



3. Скрипт контроллера игрока

Создайте скрипт CharacterController.cs и добавьте его на объект Player.

Скрипт реализует:

- перемещение игрока
- прыжок
- управление камерой
- переключение между видом от первого и третьего лица

Управление:

W A S D – движение

Space – прыжок

Мышь – вращение камеры

T – переключение камеры

Скрипт CharacterController.cs

```
using UnityEngine;
using UnityEngine.SceneManagement;

public class PlayerController : MonoBehaviour
{
    [Header("Movement")]
    public float walkSpeed = 5f;
    public float runSpeed = 9f;

    [Header("Jump")]
    public float jumpForce = 7f;
    public bool isGrounded;

    [Header("Mouse Look")]
    public Transform cameraPivot;
    public float mouseSensitivity = 2f;
    public float maxPitch = 80f;

    [Header("Character Variables")]
    public int maxHP = 100;
    public int hp = 100;

    [Header("Camera Settings")]
    public Camera mainCamera; // Main Camera (3-е лицо)
    public float thirdPersonDistance = 5f;
    public float thirdPersonHeight = 2f;
    public LayerMask cameraCollisionMask;

    [Header("First Person Settings")]
    public float firstPersonHeight = 1.7f; // Высота камеры от пола (уровень глаз)

    private Rigidbody rb;
    private float yaw;
    private float pitch;
    private Vector3 moveInput;
    private float currentSpeed;
    private bool isFirstPerson = true;
    private Camera firstPersonCamera; // Будет создана программно

    void Awake()
    {
        rb = GetComponent<Rigidbody>();
        hp = maxHP;
    }
}
```

```

camera1st.enabled = true;
camera3rd.enabled = false;

Cursor.lockState = CursorLockMode.Locked;
Cursor.visible = false;

camera3rd.transform.localPosition = tpsOffset;
}

void Update()
{
    HandleMovement();
    HandleMouseLook();
    HandleCameraSwitch();
    HandleTPSCamera();
}

void HandleMovement()
{
    isGrounded = controller.isGrounded;

    if (isGrounded && velocity.y < 0)
        velocity.y = -2f;

    float moveX = Input.GetAxis("Horizontal");
    float moveZ = Input.GetAxis("Vertical");

    Vector3 move = transform.right * moveX + transform.forward * moveZ;
    controller.Move(move * speed * Time.deltaTime);

    if (isGrounded && Input.GetButtonDown("Jump"))
        velocity.y = Mathf.Sqrt(jumpHeight * -2f * gravity);

    velocity.y += gravity * Time.deltaTime;
    controller.Move(velocity * Time.deltaTime);
}

void HandleMouseLook()
{
    float mouseX = Input.GetAxis("Mouse X") * mouseSensitivity;
    float mouseY = Input.GetAxis("Mouse Y") * mouseSensitivity;

    transform.Rotate(Vector3.up * mouseX);

    xRotation -= mouseY;
    xRotation = Mathf.Clamp(xRotation, -maxLookAngle, maxLookAngle);

    if (isFirstPerson)
        camera1st.transform.localRotation = Quaternion.Euler(xRotation, 0f, 0f);
}

```

```

void HandleCameraSwitch()
{
    if (Input.GetKeyDown(KeyCode.T))
    {
        isFirstPerson = !isFirstPerson;

        camera1st.enabled = isFirstPerson;
        camera3rd.enabled = !isFirstPerson;

        if (!isFirstPerson)
            camera3rd.transform.localPosition = tpsOffset;
    }
}

void HandleTPSCamera()
{
    if (!isFirstPerson)
    {
        Vector3 desiredPosition = tpsOffset;
        camera3rd.transform.localPosition = Vector3.Lerp(camera3rd.transform.localPosition,
desiredPosition, Time.deltaTime * cameraSmoothSpeed);
        camera3rd.transform.LookAt(transform.position + Vector3.up * 1.6f);
    }
}
}

```

4. Настройка NavMesh

Создайте объект противника:

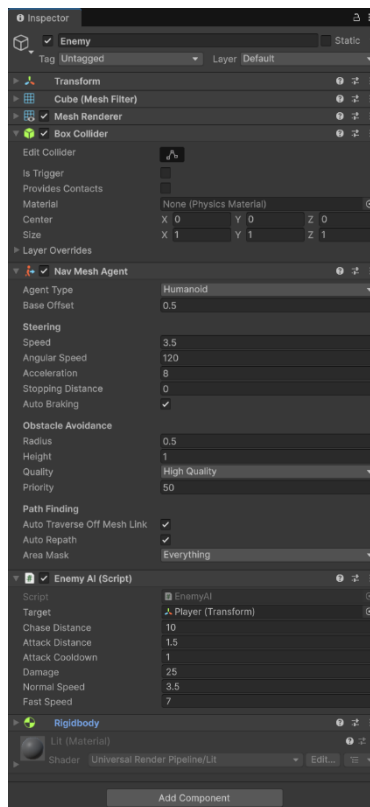
GameObject - 3D Object - Sphere

Переименуйте его в Enemy.

Добавьте компоненты:

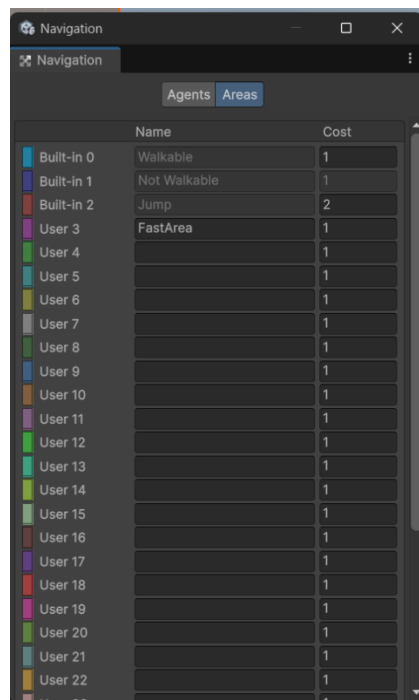
NavMeshAgent

Rigidbody



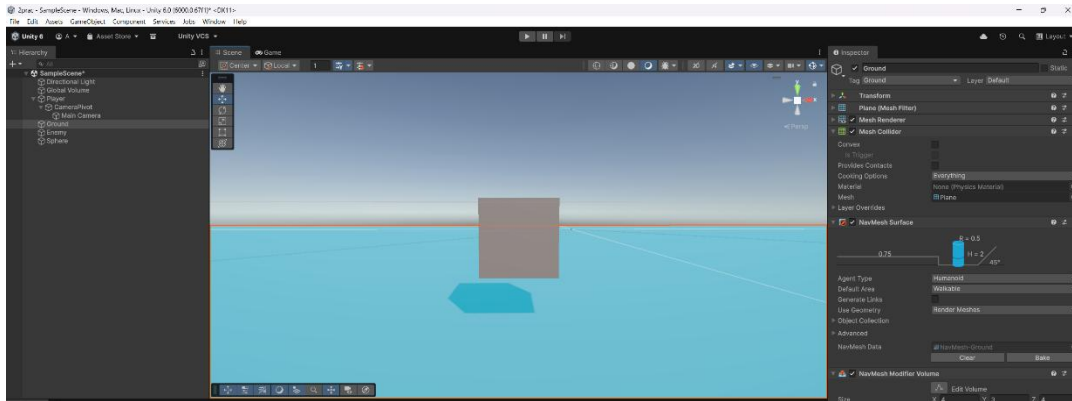
Откройте окно навигации:

Window - AI – Navigation



Выберите Plane и отметьте Navigation Static.

Нажмите Bake для генерации NavMesh.



5. Создание зон NavMesh

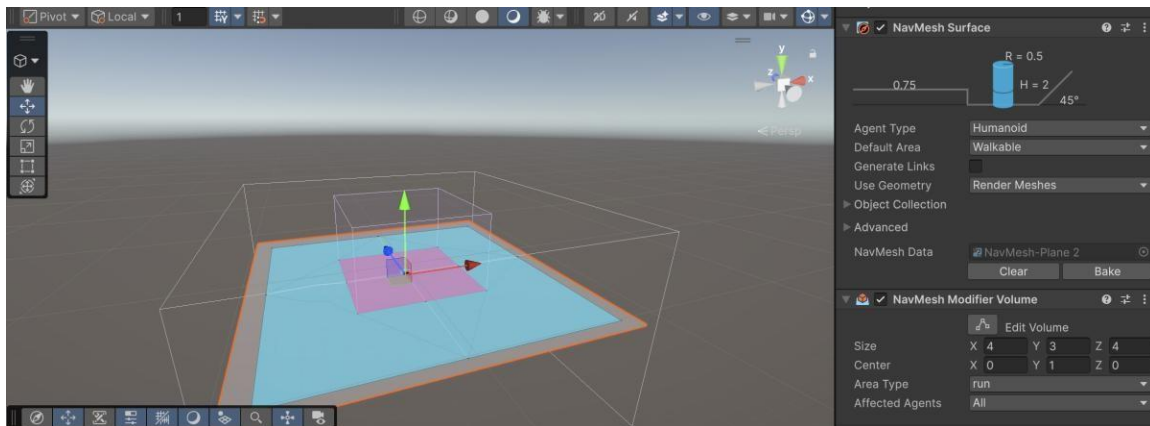
В окне Navigation откройте вкладку Areas и создайте новую область под названием run.

Назначьте разные области поверхности с помощью NavMesh Modifier Volume:

центральную часть – run

остальную часть – Walkable

После настройки снова нажмите Bake.



6. Скрипт NPC

Создайте скрипт NPCController.cs и добавьте его на объект Enemy.

Скрипт реализует:

- перемещение NPC по NavMesh

- преследование игрока

- изменение скорости в зависимости от зоны NavMesh

В Inspector у объекта Enemy в поле Player перетащите объект Player.

Скрипт NPCController.cs

```
using UnityEngine;
using UnityEngine.AI;

public class EnemyAI : MonoBehaviour
{
    public Transform target;
    public float chaseDistance = 10f;
    public float attackDistance = 1.5f;
    public float attackCooldown = 1f;
    public int damage = 10;

    public float normalSpeed = 3.5f;
    public float fastSpeed = 7f;

    private NavMeshAgent agent;
    private float nextAttackTime;

    int fastArea;

    void Awake()
    {
        agent = GetComponent<NavMeshAgent>();

        fastArea = NavMesh.GetAreaFromName("FastArea");
    }

    void Update()
    {
        if (target == null) return;

        CheckFastArea();

        float dist = Vector3.Distance(transform.position, target.position);

        if (dist <= chaseDistance)
        {
            agent.SetDestination(target.position);
        }
        else
        {
            agent.ResetPath();
        }

        if (dist <= attackDistance && Time.time >= nextAttackTime)
        {

```

```
if (NavMesh.SamplePosition(transform.position, out hit, 1f, NavMesh.AllAreas))
{
    int mask = hit.mask;

    if (mask == (1 << runArea))
        agent.speed = runSpeed;

    else if (mask == (1 << walkableArea))
        agent.speed = walkSpeed;
}
}
```